

Code Quality Report — Tick & Taka (round 2)

Bun monorepo: Hono API, Expo Android app, shared package

Project	Tick & Taka (tick-taka)
Date	2026-10-06
Commit	df29d67 (branch fix/audit-2026-10-06)
Previous audit	Round 1 at 3b5438b, score 70/100, 26 findings. 75 commits since.
Auditor role	Staff Engineer (architecture, maintainability, security, performance)
Companion reports	QA_Report.pdf and UX_Report.pdf, written in parallel by other auditors
Mode	Read-only. No source, config, dependency or git state was changed. Probe scripts ran against in-memory test databases from a scratch folder that was deleted afterwards.

1. Executive summary

72

/ 100

round 1: 70

The 75 fix commits are real work. I checked every round-1 finding against the code: 19 are fixed, 5 partly fixed, 2 not fixed, none regressed. The 401-wipes-the-queue bug is gone. Sessions are revocable, the outbox has its own storage and survives a 401, the cache is encrypted, the API binds to loopback, AI calls abort and stop at the cap, and CI now runs lint, typecheck and 568 tests on every push.

The score barely moves for one reason. Any signed-in user can take the whole API down. The `timeZone` setting accepts any string, and the hourly job loop has no per-user error handling. One bad value makes the job promise reject with nothing to catch it, and Bun exits. The catch-up run 10 seconds after start then crashes it again, so pm2 restarts it into the same crash. I reproduced this. The fix is an hour's work, and with it the codebase would sit near 80.

Severity (open findings)	Count
Critical	1
High	0
Medium	3
Low	12
Total	16 (9 new, 7 carried over)

Round-1 status	Count
Fixed	19
Partially fixed	5
Not fixed	2
Regressed	0

Top 5 issues to fix first

1. **CQ-027** **Critical** An invalid time zone in settings crashes the API process every time the job loop runs, and the loop runs 10 s after every start. Any account can trigger it, and so can the assistant's `update_settings` action.
2. **CQ-029** **Medium** `/auth/refresh` revokes the old token at once. Requests already in flight with it come back 401, and the phone can sign itself out during its own daily refresh.
3. **CQ-030** **Medium** CHECK-constraint violations return 500, and the outbox retries any 5xx forever at the head of the queue, so one bad write holds back every write behind it.
4. **CQ-028** **Medium** Encrypted storage deletes a value whenever decryption fails. That includes the case where the key could not be read for a moment, so the queued writes are lost.
5. **CQ-031** **Low** Deleting an account while the hourly job runs recreates that user's database and a backup (reproduced).

2. Scorecard

Area	R1	R2	Justification
Architecture	8	8	Clear module-per-domain API, shared zod package, typed RPC. The new handle cache with leases is a good design. One new import cycle (<code>db/schema/time.ts</code> ↔ <code>money.ts</code>) is harmless because the FK references are lazy.
Readability & maintainability	7	7	No file over 800 lines (largest is 583), no TODOs, little duplication. The mobile sheets were split, but <code>taskService</code> (378 lines) and <code>createToolRunner</code> (342 lines) both grew (CQ-020).
Type safety	8	7	Strict mode, 1 <code>any</code> , no <code>@ts-ignore</code> . Non-null <code>!</code> went from 44 to 60 and the lint rule is still off. <code>timeZone</code> is an unchecked string at a trust boundary, which is the root of CQ-027.
Error handling	5	6	The client got much better: a 401 keeps data, start-up never hangs, there is an error boundary, and logs are structured. The server job loop has no per-user isolation (CQ-027), CHECK errors are 500s (CQ-030), and storage deletes on any decrypt error (CQ-028).
Security	6	6	Big gains: revocable sessions, header-only tokens, loopback bind, a trusted-proxy check, per-user AI limits, encryption at rest, account deletion, cleartext only in dev builds. Offset by a denial of service that any user can trigger (CQ-027).
Performance	6	8	LRU handle cache, jobs yield between users, abortable AI, scoped invalidation, streamed export, <code>updated_at</code> indexes. <code>/sync/changes</code> is still unbounded (CQ-016) and the owner's usage report opens every database (CQ-034).
Data layer	7	8	40 CHECKs and the missing FKs added in a tested rebuild migration. LIKE fixed. CHECK errors are not mapped (CQ-030), and there is no clean-up for rows that break a new CHECK (CQ-032).
Tooling & DX	6	8	CI with a frozen lockfile, <code>packageManager</code> pinned, all 75 commits are Conventional Commits, README matches the deploy files. Audit advisories are unchanged (CQ-012).
Testing	6	7	185 tests became 568, mobile went from 14 to 165, and the outbox, session and storage rules are pure and tested. Nothing tests a failing user in the job loop, an invalid time zone, a refresh racing other requests, or a key-read failure.
Dependencies	7	7	Lockfile frozen in CI and deploy, Bun pinned, unused <code>expo-navigation-bar</code> removed. The same 6 advisories remain, all in build tooling.

The overall score is a weighted judgement, not an average. Security, error handling and data integrity count double because the app holds personal finance records. A verified, one-request outage keeps the score from rising with the other areas.

3. Scope & method

What was reviewed

- Every changed area named in the brief: `db/sessions.ts`, `modules/auth/*`, `middleware/auth.ts`, `lib/client-ip.ts`, `middleware/rate-limit.ts`, `middleware/request-log.ts`, `lib/log.ts`, `db/handle-cache.ts`, `db/user-registry.ts`, `db/user-files.ts`, `jobs/*`, `db/job-runs.ts`, `ai/openai-client.ts`, `ai/usage.ts`, `modules/ai/routes.ts`, `modules/ai/assistant/*`, `modules/sync/routes.ts`, migrations 0003 to 0005, `.github/workflows/check.yml`, `deploy/nginx-tick-taka.conf`.
- Mobile: `lib/auth.ts`, `http.ts`, `session.ts`, `outbox*.ts`, `encrypted-storage.ts`, `secure-storage.ts`, `query-client.ts`, `invalidation.ts`, `sync-changes.ts`, `event-stream.ts`, `user-data.ts`, `app/_layout.tsx`, `features/app/error-screen.tsx`, `app.config.ts`, `app.json`.
- Whole-repo scans: file and function sizes (TypeScript AST), type-safety counts (AST), import cycles (Tarjan SCC over 390 files), duplicate 8-line blocks, unused exports, `console.*`, swallowed catches, TODOs.
- Yardstick: README.md, PRODUCT.md, docs/IMPLEMENTATION_PLAN.md, the spec (`_raw/spec.txt`), the round-1 report, and the owner's rules: an 800-line ceiling, no barrel files, strict TypeScript, Conventional Commits.

Commands run

Command	Result
<code>bunx biome check apps packages *.json .github</code>	484 files, 0 diagnostics, exit 0 (from <code>_raw/lint.txt</code>)
<code>bun run typecheck</code>	shared, api, mobile: exit 0
<code>bun run test</code>	shared 237, mobile 165, api 166 pass; 0 fail
<code>bun test --coverage per workspace</code>	api 93.78% lines / 86.91% funcs; shared 96.07 / 81.14; mobile 92.08 / 91.93 (37 files loaded)
<code>bun audit</code>	6 advisories (2 high, 4 moderate), exit 1
AST scan (TypeScript compiler API)	135 as casts, 91 as const, 60 non-null !, 1 any; function sizes in Appendix C
Import-graph SCC	1 cycle: <code>db/schema/time.ts</code> ↔ <code>db/schema/money.ts</code>
Probe scripts (Appendix F) against <code>createTestContext()</code>	Reproduced CQ-027 (process exit), CQ-030 (500 on CHECK) and CQ-031 (deleted user's files recreated)
<code>bun -e unhandled-rejection check</code>	Bun 1.3.11 exits with code 1 on an unhandled rejection

Not verified

- **Unverified** No device run. The emulator image has no Google Play services and sign-in is Google-only. CQ-028 and CQ-029 come from reading the code. I could not measure how often the refresh race or a Keystore read failure happens on a real phone.
- **Unverified** The production server: whether `.env` sets `TRUST_PROXY=true`, whether the rsync off-site backup in the README is installed, and whether real data has rows that break the new CHECKs (CQ-032).

- No `bun outdated` (it needs the network). CQ-027 was proved through the job path. I did not run pm2 itself, so the restart loop follows from pm2's documented autorestart, not from an observed loop.

4. Round-1 findings status

Each row was checked against the code at df29d67. Line numbers refer to df29d67.

ID	Round-1 finding	Status	Evidence
CQ-001	401 deletes the offline queue; no refresh Critical	Fixed	A 401 now calls <code>expireSession()</code> , which drops only the token (<code>mobile/lib/auth.ts:139-145</code> , <code>http.ts:45-53</code>). The outbox lives under <code>tt.outbox</code> outside the query cache (<code>outbox.ts:19,38-42</code>) and pauses while <code>canSend()</code> is false. Sliding refresh: <code>POST /auth/refresh</code> , used after 24 h (<code>session.ts:15</code>). New race: CQ-029.
CQ-002	Splash hang if SecureStore fails High	Fixed	<code>readStoredSession()</code> never throws and validates the profile with zod (<code>session.ts:44-79</code>). <code>_layout.tsx:80</code> also catches. Tested in <code>test/session.test.ts</code> .
CQ-003	No error boundary Medium	Fixed	Root <code>ErrorBoundary</code> (<code>_layout.tsx:39-42</code>) shows <code>ErrorScreen</code> with Try again and Reset cached data, and keeps the outbox.
CQ-004	Bare console logging Low	Fixed	JSON lines through <code>lib/log.ts</code> , request ids and HMAC user tags (<code>middleware/request-log.ts</code>), and error-mapper branches tested (<code>test/logging.test.ts</code>). One leftover <code>console.warn</code> at <code>db/client.ts:33</code> (CQ-035).
CQ-005	Token in receipt URLs Medium	Fixed	The server reads the token from the header only (<code>middleware/auth.ts:18-19</code>). Receipts load with <code>headers: authHeaders()</code> (<code>mobile/lib/receipts.ts:66</code>).
CQ-006	Sessions not revocable Medium	Fixed	<code>sessions</code> table keyed by <code>jti</code> (<code>db/sessions.ts</code>), plus refresh, logout and revocation checks in <code>requireAuth</code> ; 8 tests in <code>sessions.test.ts</code> . Follow-ups: CQ-029, CQ-033.
CQ-007	0.0.0.0 default, spoofable limiter Medium	Fixed	<code>HOST</code> defaults to <code>127.0.0.1</code> (<code>env.ts:6</code>). <code>X-Real-IP</code> is used only with <code>TRUST_PROXY</code> and a loopback peer (<code>lib/client-ip.ts</code>). Keys are pruned every 100 attempts. 30 AI calls a minute per user (<code>ai/routes.ts:39-55</code>). Production <code>TRUST_PROXY</code> value Unverified .
CQ-008	Assistant paths built from model strings Medium	Fixed	Every id goes through the ULID <code>idSchema</code> (<code>tools.ts:196-202</code>), status must match <code>^[a-z_]+(,[a-z_]+)*\$</code> (<code>tools.ts:205-209</code>), and <code>isPlainPath()</code> refuses dot segments and encoded slashes (<code>dispatch.ts:19-24</code>). Tested in <code>assistant-paths.test.ts</code> .
CQ-009	Cleartext in release builds Low	Fixed	<code>app.json</code> sets <code>usesCleartextTraffic: false</code> . <code>app.config.ts</code> turns it on only outside EAS release profiles and refuses non-https release builds. A local release build with no <code>EAS_BUILD_PROFILE</code> still allows cleartext, which is minor.

ID	Round-1 finding	Status	Evidence
CQ-010	Unencrypted cache and queue Medium	Fixed	AES-GCM through expo-crypto with the key in SecureStore (<code>secure-storage.ts</code>), used by the cache persister and the outbox. <code>allowBackup: false</code> . New issue in the failure path: CQ-028.
CQ-011	No account deletion Medium	Fixed	<code>DELETE /auth/account</code> removes sessions, job runs, the handle, DB/WAL files, uploads and backups (<code>user-registry.ts:208-214</code> , <code>user-files.ts</code>). Settings calls it. Edge case: CQ-031.
CQ-012	6 audit advisories Low	Not fixed	The same 6 advisories (braces, node-forge, postcss-selector-parser, decode-uri-component, esbuild, uuid). CI now reports them in a non-blocking job.
CQ-013	DB handles never closed; jobs block Medium	Fixed	LRU plus idle cache (max 50, 10 min) with leases (<code>handle-cache.ts</code>), a 60 s sweep (<code>index.ts:27</code>), and jobs that <code>setImmediate</code> between users and close what they opened (<code>scheduler.ts:28,61</code>). One user's <code>VACUUM INTO</code> still runs synchronously. Exception: CQ-034.
CQ-014	AI loop not aborted Medium	Fixed	An <code>AbortController</code> is wired to the request and stream abort (<code>ai/routes.ts:122-131</code>). The signal goes into OpenAI calls and is checked between rounds and tools (<code>agent.ts:211-219</code>). The cap is checked before every call (<code>usage.ts:94-109</code>). 25 s timeout with 1 retry. The plain JSON AI routes still finish after a disconnect, which is minor.
CQ-015	Every mutation invalidates every query Low	Fixed	Route-to-key mapping in <code>invalidation.ts</code> , used by <code>scheduleRefresh(path)</code> (<code>query-client.ts:21-33</code>), with tests.
CQ-016	<code>/sync</code> unused and unbounded; <code>/export</code> in memory Low	Partially fixed	<code>/export</code> streams in pages of 500 (<code>sync/routes.ts:47-88</code>). Migration 0005 indexes <code>updated_at</code> and the phone now calls <code>/sync/changes</code> . The endpoint still returns every changed row with no limit. Carried as CQ-016.
CQ-017	LIKE escaping broken Medium	Fixed	<code>likeContains()</code> adds <code>ESCAPE '\'</code> and escapes the backslash (<code>lib/sql-like.ts</code>). Used by tasks and transactions; <code>search.test.ts</code> .
CQ-018	No CHECKs, no self-reference FKs Medium	Fixed	<code>0004_constraints.sql</code> rebuilds 16 tables with 40 CHECKs and FKs on <code>parent_id</code> , <code>goal_id</code> , <code>next_id</code> . Dangling links are cleared first, and the migration is tested on a populated DB (<code>migrations.test.ts</code>). Follow-ups: CQ-030, CQ-032.
CQ-019	Backups on the same disk Medium	Partially fixed	Each backup is integrity-checked before it is kept (<code>backup.ts:18-61</code>). The server warns at start when backups share the data disk (<code>index.ts:21-25</code>), and the README gives an <code>rsync</code> cron. Nothing copies backups off the box automatically. Carried as CQ-019.

ID	Round-1 finding	Status	Evidence
CQ-020	Oversized components and closures Medium	Partially fixed	TaskSheet went from 443 to 292 lines and TransactionSheet from 373 to 284; use-quick-add was split. taskService grew from 333 to 378, createToolRunner from 310 to 342, tools.ts from 487 to 556 and quick-add.ts from 475 to 583. Carried as CQ-020.
CQ-021	TZ/week fallbacks repeated Low	Fixed	mobile/lib/user-time.ts. No "Asia/Dhaka" or ?? 6 literals remain in apps/mobile/src.
CQ-022	! allowed; unchecked client casts Low	Not fixed	biome.json:23 still has noNonNullAssertion: "off". The count went from 44 to 60, and the same sites remain (planning.ts:50,53,69, today/service.ts:119). api.ts:49,66 still cast JSON. The profile is now zod-checked. Carried as CQ-022.
CQ-023	No CI Medium	Fixed	.github/workflows/check.yml: frozen install, bun run check, non-blocking bun audit. The README deploy uses --frozen-lockfile.
CQ-024	Mobile data path untested Medium	Partially fixed	31 mobile test files, including outbox-queue, outbox-policy, session, http-auth and encrypted-storage.auth.ts (the refresh and expire flow), secure-storage.ts and sync-changes.ts are untested, and that is where CQ-028 and CQ-029 sit. Carried as CQ-024.
CQ-025	Deploy config drift Low	Partially fixed	One host (tick.mehedismathacademy.com) and port 3003 across README, eas.json and Nginx. /ai/ has proxy_read_timeout 120s and proxy_buffering off. The TLS server block is still produced by certbot on the server and is not in the repo. Carried as CQ-025.
CQ-026	Dependency hygiene Low	Fixed	"packageManager": "bun@1.3.11"; expo-navigation-bar removed (9b1d346). expo-system-ui stays and is needed for userInterfaceStyle on Android. zod 3 remains only through @expo/cli (build time).

5. Findings

Grouped by category, sorted by severity. New findings are numbered from CQ-027. Carried-over findings keep their round-1 IDs. Effort: S < 1 h, M < 1 day, L > 1 day.

5.1 Reliability & data integrity

CQ-027 · An invalid time zone setting crashes the API in a restart loop

Critical · Reliability / security (denial of service) · Effort S · New · Reproduced

Where

packages/shared/src/schemas/settings.ts:24, apps/api/src/jobs/scheduler.ts:36-65, 72-86, apps/api/src/lib/user-time.ts:14, apps/api/src/modules/ai/assistant/tools.ts:517-522

What's wrong

timeZone is `z.string()`, so `PATCH /settings {"timeZone":"Not/AZone"}` is saved with a 200. After that, `localParts()` throws `RangeError` for this user. Every screen that reads their time (`/today` and others) returns 500.

The bigger problem is the job loop. In `runHourlyJobs` only `job.run()` sits inside a try/catch. `deps.users.lease(user)`, `readSettings()` and `localParts()` are outside it, so one user's error rejects the whole run. Neither caller handles that rejection: the catch-up is `setTimeout(() => void tick(), 10_000)`, and `croner` runs with `catch: false` and does not await `_trigger()`. Bun 1.3.11 exits on an unhandled rejection. pm2 restarts the process, the catch-up runs 10 s later, and it crashes again.

Sign-up is open to any Google account. The assistant can also hit this by accident:

`update_settings` passes the model's fields straight to `PATCH /settings`, so "set my time zone to Dhaka" can save "Dhaka". A user DB that fails to open, for example a failed migration (CQ-032), takes the same path through `lease()`.

Evidence

```
// scheduler.ts:40-46 (no catch around the per-user body)
const lease = deps.users.lease(user);           // throws if the DB can't open
try {
  runAsUser({ user, data }, () => {
    const timeZone = readSettings(data.db).timeZone;
    const { hour } = localParts(now, timeZone); // RangeError: invalid time zone
  })
} catch {
  // scheduler.ts:80-81
  const job = new Cron("5 * * * *", { name: "per-user-jobs" }, tick);
  const catchUp = setTimeout(() => void tick(), 10_000);
}
```

```
$ bun probe-tz.ts      (second user, in-memory test app)
PATCH settings bad tz: 200
GET /today for that user: 500 {"error":{"code":"internal_error",...}}
runHourlyJobs REJECTED: RangeError: invalid time zone: Not/AZone
$ bun probe-crash.ts   (same setup, then startJobs(deps))
  at localParts (packages/shared/src/dates.ts:50:17)
  at runHourlyJobs (apps/api/src/jobs/scheduler.ts:44:7)
exit=1                (about 10 s after start)
```

Why it matters

One request from any account takes the API down for every user, and it stays down across restarts until someone fixes that user's settings row by hand in SQLite. The affected user's own app also breaks, because the phone computes dates with the same value.

Recommendation

Fix all three layers.

```
// shared/schemas/settings.ts: only real IANA zones
const isTimeZone = (tz: string) => { try { new Intl.DateTimeFormat("en", { timeZone: tz });
return true; } catch { return false; } };
timeZone: z.string().refine(isTimeZone, "Unknown time zone").default(DEFAULT_TIME_ZONE),
// readSettings already falls back field by field, so stored bad values heal on read.

// scheduler.ts: one user's failure is logged and skipped
for (const user of deps.users.list()) {
  await yieldToRequests();
  try { runJobsFor(deps, user, now); } // lease + runAsUser + release
  catch (error) { log("error", "user jobs failed", { user: userTag(...),
...errorFields(error) }); }
}
// and nothing can reject unseen
running ??= runHourlyJobs(deps).catch((e) => log("error", "job run failed",
errorFields(e))).finally(...);
process.on("unhandledRejection", (e) => log("error", "unhandled rejection", errorFields(e)));
```

Add tests: PATCH with a bad zone returns 400, and `runHourlyJobs` still runs user B's jobs when user A's DB throws.

CQ-030 · CHECK violations return 500, and the outbox retries any 5xx forever at the head of the queue

Medium · Error handling / offline sync · Effort M · New · Reproduced (API side)

Where

`apps/api/src/middleware/error-handler.ts:17-31`, `apps/api/src/modules/events/routes.ts:48-50`, `packages/shared/src/schemas/money.ts:264-273`, `apps/mobile/src/lib/outbox-policy.ts:73`, `apps/mobile/src/lib/outbox-queue.ts:160-199`

What's wrong

Migration 0004 added 40 CHECKs, but the error mapper only knows FOREIGNKEY (400) and UNIQUE/PRIMARYKEY (409). A CHECK failure becomes `internal_error` 500. Partial PATCH schemas cannot check two fields against each other. `eventUpdateSchema` accepts `endsOn` alone, the route does not compare it with the stored `startsOn`, and the database refuses the row. On the phone, the outbox classifies every 5xx as `retry`, retries forever (capped at 60 s), and only ever sends `entries[0]`. A write the server always rejects with 5xx stays at the head, and every write queued after it waits behind it.

Evidence

```
$ bun probe-check.ts
POST /events {startsOn:"2026-10-10", endsOn:"2026-10-12"} -> 201
PATCH /events/:id {endsOn:"2026-10-01"} -> 500 {"error":{"code":"internal_error",...}}
// outbox-policy.ts:73
if (status >= 500 || status === 408 || status === 429) return "retry";
// outbox-queue.ts:164-165
while (this.entries.length > 0 && this.deps.canSend()) {
  const entry = this.entries[0]!;
```

Why it matters

The event sheet clamps `endsOn` (`event-sheet.tsx:56`), so this exact payload does not come from today's UI. The pattern is general, though. Any deterministic 5xx, whether an unmapped constraint or a bug in one service, quietly stops all syncing on that phone. The sync bar shows a count, but nothing lets the user see or drop the stuck write. Unsynced writes then pile up behind it, which the offline queue exists to prevent.

Recommendation

Map `SQLITE_CONSTRAINT_CHECK` and `_NOTNULL` to 400 `invalid_value`, so the outbox rejects the write and tells the user. Check cross-field ranges on PATCH against the stored row, as `time-entries/service.ts:174-176` already does. In the outbox, mark an entry as stuck after about 8 failed 5xx attempts, keep sending the entries after it, and list stuck writes in Settings with Retry and Discard. Add a test with one poison entry followed by a good one.

CQ-028 · Encrypted storage deletes the queued writes when the key can't be read

Medium · Data integrity (client) · Effort S · New · **Unverified on device** (how often the trigger happens)

Where

`apps/mobile/src/lib/encrypted-storage.ts:28-58`, `apps/mobile/src/lib/secure-storage.ts:11-35`, `apps/mobile/src/lib/outbox.ts:38-42`, `apps/mobile/src/lib/outbox-queue.ts:85-102`, `236-245`

What's wrong

`getItem` treats any error from `cipher.decrypt` as "the key is gone for good" and deletes the value. `decrypt` awaits `storageKey()`, which rejects whenever `SecureStore.getItemAsync` fails. That is the same Keystore read failure round 1 flagged in CQ-002, and here it can be temporary. A failed key read at start-up therefore deletes `tt.outbox` and the cached screens. Nobody is told.

Writes are protected the other way round: `setItem` falls back to plain text when the cipher fails, so writes are never lost but reads are. Even if `getItem` threw instead of deleting, `OutboxQueue.load` catches the error, marks itself loaded and persists an empty queue on the next write, which overwrites the unread data.

There is also a small race. Reading a plain value starts a re-seal (`void cipher.encrypt(raw).then(setItem)`) that is not ordered against later `setItem` calls, so it can write the older value over a newer one.

Evidence

```
// encrypted-storage.ts:33-45
if (!raw.startsWith(SEALED_PREFIX)) {
  void cipher.encrypt(raw).then((sealed) => base.setItem(key, SEALED_PREFIX +
sealed)).catch(() => {});
  return raw;
}
try { return await cipher.decrypt(raw.slice(SEALED_PREFIX.length)); }
catch { await base.removeItem(key).catch(() => {}); return null; } // also on "key
unreadable"
// secure-storage.ts:13, 32
const saved = await SecureStore.getItemAsync(KEY_NAME);
const bytes = await aesDecryptAsync(AESSealedData.fromCombined(sealed), await storageKey());
```

Why it matters

The queued writes are the data round 1's Critical fix set out to protect. A real Keystore reset makes them unreadable anyway. A temporary read failure should not destroy them.

Recommendation

Delete only when the key loaded and decryption failed (a wrong key or a tag mismatch). If the key cannot be loaded, throw a `KeyUnavailableError` and keep the sealed value. In `OutboxQueue.load`, on a load error, keep new writes in memory and do not persist until a read succeeds. A retry on the next foreground can merge them, the same way `discardSaved` already works. Drop the background re-seal; the next `setItem` seals the value anyway. Add tests where `storageKey` rejects once and then succeeds.

CQ-031 · Deleting an account during the hourly job run recreates its database and a backup

Low · Data lifecycle / privacy · Effort S · New · Reproduced

Where

apps/api/src/jobs/scheduler.ts:37-40, apps/api/src/db/user-registry.ts:171-181, 208-214

What's wrong

`runHourlyJobs` takes `deps.users.list()` once and then yields before each user. If `DELETE /auth/account` lands during the run, the loop still reaches the deleted user. `lease()` then calls `data()`, which opens (and so creates) a fresh `users/<id>.db` and seeds it. The backup job then writes `backups/users/<id>/app-YYYY-MM-DD.db`, and `job_runs` gets rows for a user who no longer exists.

Evidence

```
$ bun probe-delete.ts (runHourlyJobs started, then users.remove(gone))
db file right after delete: false
db file after job run: true
backups for deleted user: [ "app-2026-10-04.db" ]
users row: gone
```

Why it matters

The recreated files hold only seeded defaults, so no personal data comes back. They are orphans that nothing will ever clean up, though, and they make "we delete your account" less exact.

Recommendation

Re-check `deps.users.find(user.id)` after the yield, and skip the user if it is gone. Give `data()` an `{ create: false }` mode for existing users, so it never creates a file for a missing registry row.

CQ-032 · Migration 0004 has no clean-up for rows that break the new CHECKs

Low · Data layer / migrations · Effort S · New · **Unverified** (no production data)

Where

`apps/api/drizzle/0004_constraints.sql:1-7` (it clears only dangling FKs),
`apps/api/src/db/client.ts:18-35`

What's wrong

Each rebuild copies rows with `INSERT INTO __new_x SELECT ... FROM x`. If one existing row breaks a new CHECK, the whole migration aborts and `openDatabase` throws. That user's every request then returns 500, and through CQ-027 the job loop rejects too. At least one way to create such a row existed before this release and still exists: an event PATCHed through the API with `ends_on < starts_on` (see CQ-030). `migrations.test.ts` covers a populated database with valid rows only.

Why it matters

Migrations run lazily the first time each user's database is opened, so a bad row shows up as "the app is broken for me" with nothing in the logs that points at the migration.

Recommendation

Normalise known violators before the rebuild, for example `UPDATE events SET ends_on = starts_on WHERE ends_on < starts_on`. Run the migration against copies of the production backups before deploying. Log the migration error with the user tag when `openDatabase` fails.

CQ-019 · Off-site backups still depend on a manual cron (carried over)

Low (was Medium) · Data layer / operations · Effort M · **Partially fixed** · **Unverified** (server)

Where

`apps/api/src/index.ts:21-25`, `apps/api/src/jobs/backup.ts:74-77`, `README.md:311`

What's wrong

The server now warns at start-up when backups share the data disk, and the README suggests `rsync` to another host. The code still writes backups only to local disk, unencrypted. Whether the cron is installed is unknown.

Why it matters

If the VPS volume is lost, the live data and all 14 backups go with it unless the cron exists.

Recommendation

Confirm the `rsync` cron on the server and keep the start-up warning. If you want it in code, add an optional encrypted push with `age` to S3 or B2 after `runBackup`, and add a restore check to the README.

5.2 Security & sessions

CQ-029 · Token refresh revokes the old session at once; concurrent 401s can sign the phone out

Medium · Security / session handling · Effort S · New · **Unverified on device** (timing)

Where

apps/api/src/modules/auth/routes.ts:60-64, apps/mobile/src/lib/auth.ts:81-91, 139-173, apps/mobile/src/lib/http.ts:45-53, apps/mobile/src/lib/event-stream.ts:39

What's wrong

The refresh and the screen refetches both start on `AppState "active"`. After 24 h, `refreshSessionIfStale()` posts to `/auth/refresh`, and `focusManager` refetches every stale query with the same old token at the same moment. The server revokes the old session as soon as it has issued the new one. Any of those GETs whose `requireAuth` runs after the revoke gets 401 `session_expired`.

On the phone, `saveSession` awaits three Keystore writes before it switches `tokenStore`. A 401 for the old token that arrives in that window passes the ky hook's "is this the current token?" check, so `expireSession()` runs and the login screen appears. If this happens before line 161, the refresh's own `tokenStore.get() === token` check fails and the new token is thrown away. If it happens during `saveSession`, the delete of `TOKEN_KEY` can run after the write, and the next cold start opens on login.

`event-stream.ts:39` calls `reportUnauthorized()` on any 401 without the token check.

Evidence

```
// api auth/routes.ts:60-64
.post("/refresh", requireAuth(deps), async (c) => {
  const { token, expiresAt } = await issue(user);
  if (sessionId) deps.users.sessions.revoke(sessionId);    // old token dead now
// mobile auth.ts:83-89
await SecureStore.setItemAsync(TOKEN_KEY, session.token);
await SecureStore.setItemAsync(PROFILE_KEY, JSON.stringify(session.user));
await SecureStore.setItemAsync(ISSUED_KEY, String(now));
...
tokenStore.set(session.token);                            // only now
// mobile auth.ts:161
if (session.success && tokenStore.get() === token) await saveSession(session.data);
```

Why it matters

The queue is safe, because the outbox survives expiry. The likely symptom is an occasional "sign in again" with Google right when the app opens, about once a day for an active user. That is the friction sliding sessions were meant to remove. `http-auth.test.ts` covers a 401 that arrives after the new token is in place, not during the switch.

Recommendation

On the server, give the old session a short grace period after refresh: set `expires_at = now + 60 s` instead of `revoked_at`, and revoke at once only on logout. On the phone, set `tokenStore` before persisting. In the 401 hook, ignore 401s while `refreshing` is pending and re-check once it settles. Use the same token check in `event-stream.ts`. Add a test for a 401 that arrives mid-refresh.

CQ-033 · Tokens issued before sessions existed stay valid and can't be revoked

Low · Security · Effort S · New

Where

`apps/api/src/middleware/auth.ts:31-43`, `apps/api/src/modules/auth/routes.ts:66-70`

What's wrong

A JWT with no `jti` skips the session lookup. Logout cannot revoke it (`sessionId` is undefined), so it works until its 30-day `exp`. This is a deliberate migration path and it is tested (`sessions.test.ts:113`), but nothing marks when it should be removed.

Why it matters

For up to 30 days after the sessions release reached the server, "sign out" does not end those sessions. After that, the branch is dead code that weakens the auth path.

Recommendation

Add a dated comment and remove the branch 30 days after the deploy. If one forced re-sign-in is acceptable, reject `jti`-less tokens now with `session_expired`; the outbox survives it.

CQ-012 · The same 6 dependency advisories (carried over)

Low · Security / dependencies · Effort S · Not fixed

Where

`docs/audit/2026-10-06-r2/_raw/audit.txt`, `.github/workflows/check.yml:19-28`

What's wrong

2 high (braces through tailwindcss, expo, react-native and worklets; node-forge through expo) and 4 moderate (postcss-selector-parser, decode-uri-component through expo-router, esbuild through drizzle-kit and tailwindcss, uuid). All are in build or CLI tooling. Only decode-uri-component may be bundled, and the risk there is a DoS on malformed deep links. CI now reports them without blocking.

Why it matters

Low runtime exposure. Left alone, the non-blocking job turns into noise that people learn to skip.

Recommendation

Run `bun update` inside the current ranges and re-audit. Add `overrides` for braces and uuid if Expo pins them. Record any advisories you accept in the README so the CI output means something.

5.3 Performance

CQ-034 · The owner's usage report opens every user's database in one request

Low · Performance / scalability · Effort S · New

Where

`apps/api/src/modules/ai/usage-report.ts:67-79`

What's wrong

For the owner, `GET /ai/usage` calls `deps.users.data(user)` for every user, synchronously and without a lease. Each first open runs migrations and seeding on the event loop. The handle cache's over-limit sweep closes only handles idle for 30 s or more, so a single call can push the open count far past 50. It keeps 2 to 3 file descriptors per database, with WAL, until the next sweep.

Why it matters

It undoes the CQ-013 fix exactly where the user count is largest. With open sign-up, a few hundred accounts would come near the default 1024 file-descriptor limit and block requests while the report runs.

Recommendation

Use `lease(user)` plus `release({ closeIf0pened: true })` as the jobs do, with a yield between users. Better still, record per-user monthly AI cost in `users.db` when usage is logged, so the report needs no user database at all.

CQ-016 · /sync/changes still returns every changed row with no limit (carried over)

Low · Performance · Effort S · Partially fixed

Where

`apps/api/src/modules/sync/routes.ts:16-37`, `apps/mobile/src/lib/sync-changes.ts:38-62`

What's wrong

The endpoint now has a client and indexes, but it still reads every row with `updated_at > since` from 20 tables into one JSON body. `pullChanges` only needs the table names, and the assistant recovery needs only rows from a short window. A phone that comes back after weeks offline, or any caller sending `since=0`, gets the whole database.

Why it matters

Memory and payload grow with history. Data volumes are personal-scale today.

Recommendation

Add `?tables=only` (counts per table) for `pullChanges`, and cap rows per table, for example 500 with a `more: true` flag that makes the phone refresh everything.

5.4 Maintainability & type safety

CQ-020 · The API's two largest closures keep growing (carried over)

Low (was Medium) · Maintainability · Effort M · Partially fixed

Where

`apps/api/src/modules/tasks/service.ts:58-435` (taskService, 378 lines),
`apps/api/src/modules/ai/assistant/tools.ts:215-556` (createToolRunner, 342 lines; its `act` switch covers lines 380-526), `packages/shared/src/quick-add.ts` (583 lines)

What's wrong

The mobile sheets were split along their real seams, and TaskSheet went from 443 to 292 lines. The API side went the other way. `createToolRunner` gained money drafts, and `taskService` gained lossless undo. Both are under the 800-line ceiling. `tools.ts` is the assistant's security boundary, and it is the least covered API file (73.89% lines; 412-483 and 491-522 uncovered).

Why it matters

Changes to the assistant's write rules land in one 342-line closure with 10 action cases. Reviewing such changes, and testing each case, gets harder with every feature.

Recommendation

Move each `act` case into a handler table in a sibling `assistant/actions.ts` (`Record<ActionName, (ctx, id, raw) => Promise<unknown>>`). That is a move, not a rewrite. Do the same for the undo and bulk-move parts of `taskService`. Add tests for the uncovered `act` cases.

CQ-022 · Non-null assertions are still allowed and growing; client JSON is still cast (carried over)

Low · Type safety · Effort M · Not fixed

Where

`biome.json:23; 60 !` sites (top: `shared/quick-add.ts` 9, `assistant/fields.ts` 5, `ai/planning.ts` 3, `tasks/service.ts` 3); `mobile/src/lib/api.ts:49,66`

What's wrong

The same domain-field assertions as in round 1 remain (`t.doAt!` at `planning.ts:50,53,69`; `debt.remindAt!` at `today/service.ts:119`; `task.doAt!` at `tasks/service.ts:364,417,418`), and the count rose from 44 to 60. `unwrap` and `send<T>` still cast server JSON without checking it. Typed RPC covers the fresh-response case. Cached data restored from an older build is never re-validated, but the error boundary now limits the damage.

Why it matters

Each `!` depends on a filter somewhere else. If that filter changes, the result is `NaN` times rather than a compile error.

Recommendation

Set the rule to `warn` and narrow with type guards (`.filter((t): t is Task & { doAt: number } => t.doAt !== null)`). Bump the persister `buster` whenever a response shape changes.

CQ-035 · Dead schema and small hygiene leftovers

Low · Maintainability / dead code · Effort S · New

Where

```
apps/api/src/db/schema/money.ts:227 ( smsImports), db/tables.ts:11,47,  
drizzle/0004_constraints.sql:173-192, drizzle/0005_sync_updated_at.sql:5,  
mobile/src/lib/sync-paths.ts:34; packages/shared/src/finance.ts:127;  
packages/shared/src/schemas/common.ts:38; apps/api/src/db/client.ts:33;  
db/schema/time.ts:11 ↔ money.ts:11
```

What's wrong

- SMS capture was dropped (IMPLEMENTATION_PLAN.md:11), yet `sms_imports` is still created, rebuilt with CHECKS in 0004, indexed in 0005, exported and synced.
- `monthlySetAside` and `paginationQuerySchema` are referenced nowhere, not even in tests. About 40 more exports are used only inside their own file.
- `openDatabase` logs FK problems with `console.warn` instead of `log()`, so they skip the JSON format.
- The new FK from `tasks.goal_id` creates the repo's only import cycle (`time.ts ↔ money.ts`). It is safe because `.references(() => ...)` is lazy.

Why it matters

Small, but dead tables cost a rebuild in every migration that touches them, and every new reader has to work out that they do nothing. That goes against "Every feature earns its place" (spec).

Recommendation

Drop `sms_imports` in the next migration, delete the two unused exports, and route the FK warning through `log("warn", ...)`. If you want the cycle gone, move `goals` to its own schema module.

5.5 Tooling & testing

CQ-024 · The riskiest new client and job paths have no tests (carried over)

Low (was Medium) · Testing · Effort M · **Partially fixed**

Where

`apps/mobile/src/lib/auth.ts`, `secure-storage.ts`, `sync-changes.ts` (none loaded by any test);
`apps/api/src/jobs/scheduler.ts:71-83` uncovered; `apps/mobile/src/lib/http.ts` 46% of functions

What's wrong

The owner moved the rules into pure modules and tested them well: `outbox-queue`, `outbox-policy`, `session`, `encrypted-storage`. The glue around them has no tests, and every new bug in this report sits in that glue. Nothing covers the refresh-versus-401 ordering (CQ-029), a key-load failure through the real `secureStorage` (CQ-028), a failing user inside `runHourlyJobs` (CQ-027), or `startJobs` itself.

Why it matters

A green CI gives confidence the rules are correct, but none about how they are wired.

Recommendation

Test the cases with Given/When/Then.

```
Given user A's settings hold an invalid time zone, when runHourlyJobs runs, then user B's
backup is written and the promise resolves.
Given a refresh is in flight, when a 401 for the old token arrives, then tokenStore keeps a
token and expireSession is not called.
Given storageKey() rejects once, when the outbox loads, then tt.outbox is still in storage
and loads on the next try.
```

For `auth.ts`, inject `SecureStore` and the `ky` instance through a small factory, the way `OutboxQueue` takes its deps.

CQ-025 · The live TLS server block is not in the repo (carried over)

Low · Tooling / operations · Effort S · Partially fixed

Where

`deploy/nginx-tick-taka.conf:1-36`

What's wrong

Host, port and AI timeouts now agree across files. The file still listens on port 80 only and expects certbot to rewrite it on the server, so the config actually serving traffic (TLS, redirect, HSTS) is not version-controlled. `location /` also keeps Nginx's default 60 s read timeout, which a slow `/export` stream could reach.

Why it matters

Rebuilding the server means re-running certbot and hoping the output matches, and nobody reviews changes made on the box.

Recommendation

Copy the post-certbot file back into `deploy/` with the certificate paths, add HSTS, and give `/export` the same 120 s timeout.

6. Strengths: keep these

- **Fixes that follow the recommendations closely and come with tests.** All 75 commits are Conventional Commits. Test count went from 185 to 568, and 46 new test files cover exactly the round-1 issues: sessions, handles, limits, logging, search, constraints, migrations, account deletion and assistant paths.
- **Pure rule modules on the phone.** `outbox-policy.ts`, `session.ts`, `encrypted-storage.ts`, `invalidation.ts` and `sync-paths.ts` hold the decisions with no React Native imports, so they test fast and read clearly. `classifyFailure` and `followCreatedRecords` are careful work.
- **Handle cache with leases.** `handle-cache.ts` is small and correct. Leases keep the SSE assistant and the streamed export safe from the sweep, and in-memory test databases are never closed.
- **Assistant hardening.** ULID-only ids, a status grammar, `isPlainPath()`, the cap checked before every model call, abort on disconnect, and money changes drafted for the user to confirm.
- **Careful logging.** One JSON line per request with a request id. Users appear as an HMAC tag. Query strings, bodies and tokens are never logged. Caller-supplied request ids must match a strict pattern.
- **Migration discipline.** 0004 clears dangling links before adding FKs, recreates every index, and is tested on a database migrated to 0003 and filled with realistic rows.
- **Backups that check themselves.** `VACUUM INTO` writes to a `.partial` file, `PRAGMA integrity_check` runs on the copy, and only then is it renamed and older ones trimmed. A missed nightly job catches up on the next tick.
- **Unchanged strengths from round 1.** One SQLite file per user, integer money, zod at every boundary, typed RPC from server to phone, no secrets in git, no barrel files, no file over 800 lines (largest 583), 0 TODO/FIXME, little duplication (10 small 8-line clusters across 390 files).

7. Action plan

Refactoring roadmap

When	Items
Now (this week)	CQ-027: validate <code>timeZone</code> , isolate each user in the job loop, catch the tick, log unhandled rejections. Deploy this first. · CQ-029: 60 s grace on refresh, and ignore 401s during a refresh · CQ-028: keep sealed data when the key can't be read · CQ-030: map CHECK to 400 and validate the event range on PATCH
Next (this month)	CQ-030: stuck-write handling in the outbox · CQ-024: tests for the cases above · CQ-031: re-check the user before each job · CQ-032: pre-clean plus a dry run against production backups · CQ-034: lease and yield in the usage report · CQ-033: date-stamp the jti-less branch
Later	CQ-020: action-handler table for the assistant · CQ-022: <code>!</code> rule to warn, type guards · CQ-016: bounded <code>/sync/changes</code> · CQ-035: drop <code>sms_imports</code> and dead exports · CQ-019: automate the off-site copy · CQ-025: commit the TLS block · CQ-012: dependency refresh

Prioritised table

Order	ID	Title	Severity	Effort
1	CQ-027	Invalid time zone crashes the API in a restart loop	Critical	S
2	CQ-029	Refresh revokes the old token at once; concurrent 401 signs out	Medium	S
3	CQ-028	Encrypted storage deletes the queue when the key can't be read	Medium	S
4	CQ-030	CHECK violations are 500; a 5xx write blocks the outbox forever	Medium	M
5	CQ-024	Glue code around the new rules is untested	Low	M
6	CQ-031	Deleted account recreated by the job run	Low	S
7	CQ-032	No pre-clean for rows that break the new CHECKs	Low	S
8	CQ-034	Usage report opens every user's database	Low	S
9	CQ-033	Pre-session tokens can't be revoked	Low	S
10	CQ-019	Off-site backup is a manual cron	Low	M
11	CQ-016	/sync/changes unbounded	Low	S
12	CQ-020	taskService and createToolRunner growing	Low	M
13	CQ-022	Non-null assertions and unchecked casts	Low	M
14	CQ-035	Dead sms_imports schema and small leftovers	Low	S
15	CQ-025	TLS server block not in the repo	Low	S
16	CQ-012	6 tooling advisories	Low	S

8. Appendix

A. Raw command output (trimmed)

```
$ bunx biome check apps packages *.json .github
Checked 484 files in 284ms. No fixes applied.                                exit 0
$ bun run typecheck
@tick-taka/shared typecheck: Exited with code 0
@tick-taka/api typecheck: Exited with code 0
@tick-taka/mobile typecheck: Exited with code 0
$ bun run test
@tick-taka/shared test: 237 pass, 0 fail, 486 expect() (12 files)
@tick-taka/mobile test: 165 pass, 0 fail, 453 expect() (31 files)
@tick-taka/api test: 166 pass, 0 fail, 621 expect() (26 files) [6.58s]
$ bun test --coverage (All files: % Funcs | % Lines)
api      86.91 | 93.78  lowest: ai/ask.ts 62.68, ai/assistant/tools.ts 73.89,
jobs/scheduler.ts 76.36, categories/routes.ts 79.25
shared  81.14 | 96.07  tiki.ts 68.52, quick-add.ts 88.75
mobile  91.93 | 92.08  http.ts 46.15 funcs, quick-add-requests.ts 72.90 lines
$ bun audit
braces <=3.0.3          high      via tailwindcss, expo, react-native, react-native-worklets
node-forge <=1.4.0       high      via expo
postcss-selector-parser moderate via tailwindcss
decode-uri-component     moderate via expo-router
esbuild <=0.24.2         moderate via drizzle-kit (api), tailwindcss
uuid <11.1.1             moderate via expo, expo-sharing, expo-splash-screen
6 vulnerabilities (2 high, 4 moderate)                                exit 1
$ bun -e '(async()=>{await null; throw new Error("boom")}()).finally(()=>{})'
error: boom                                                            exit 1 (Bun 1.3.11)
```

B. Code health counts (source only: apps/api/src, apps/mobile/src, packages/shared/src; 390 files)

Pattern	R1	R2	Notes / locations
any	1	1	api/src/lib/crud.ts:45, with a justified biome-ignore
@ts-ignore / @ts-expect-error	0	0	
biome-ignore	4	7	crud.ts:44; chat-composer.tsx:66; message-bubble.tsx:24,65,211; habit-list.tsx:115; use-notification-scheduler.ts:107. All have a reason.
as X casts (AST, excl. as const)	~187 (regex)	135	Top: assistant/tools.ts 15, lib/resource-routes.ts 10, lib/crud.ts 9, mobile/lib/outbox-policy.ts 6. as const : 91. The methods differ, so don't compare the two numbers directly.
as unknown as	1	1	mobile/src/lib/receipts.ts:51 (RN FormData file)
Non-null !	44	60	Rule off. Top: shared/quick-add.ts 9, assistant/fields.ts 5, ai/planning.ts 3, tasks/service.ts 3

Pattern	R1	R2	Notes / locations
TODO / FIXME / HACK	0	0	
<code>console.*</code>	5	5	log.ts:7,8 (the sink); db/client.ts:33; mobile use-notification-scheduler.ts:89, sync-changes.ts:58
<code>.catch(() => {}) / empty catch {}</code>	20 / 2	27 / 2	Mostly best-effort calls. Notable: <code>outbox-queue.ts:244</code> drops save failures with no signal.
Runtime import cycles	0	1	db/schema/time.ts ↔ money.ts (lazy FK thunks, safe)
Files over 800 lines	0	0	Largest source file: shared/quick-add.ts 583. Largest test file: api/test/money.test.ts 548.
Barrel index files	0	0	index files are entry points (api/src/index.ts, mobile/index.ts) or routes only
Duplicate 8-line blocks	n/a	10 clusters	All small JSX or query fragments. None crosses API and mobile.
Exports referenced nowhere	n/a	2 values, 10 types	monthlySetAside, paginationQuerySchema; unused type aliases in shared/schemas
Virtualised lists	1 / 35	1	transaction-list.tsx (SectionList). Server lists are capped. No change.

C. Largest functions and files (TypeScript AST)

Lines	Function	R1	Lines	Largest source files
378	taskService (api/tasks/service.ts:58)	333	583	packages/shared/src/quick-add.ts
342	createToolRunner (assistant/tools.ts:215)	310	556	api/src/modules/ai/assistant/tools.ts
292	TaskSheet (tasks/task-sheet.tsx:33)	443	484	packages/shared/src/recurrence.ts
284	TransactionSheet (money/transaction-sheet.tsx:35)	373	435	api/src/modules/tasks/service.ts
260	InsightsScreen (review/insights-screen.tsx:43)	260	379	api/drizzle/0004_constraints.sql
229	RecurringForm (money/recurring-sheet.tsx:49)	238	324	mobile/src/features/tasks/task-sheet.tsx
211	WeeklyReview (review/weekly-review.tsx:26)	198	318	mobile/src/features/money/transaction-sheet.tsx
209	useAssistant (assistant/use-assistant.ts:65)	n/a	315	packages/shared/src/schemas/money.ts
205	PlanHome (plan/plan-home.tsx:24)	205	303	packages/shared/src/defaults.ts
201	MoneyHome (money/money-home.tsx:31)	n/a	302	mobile/src/features/review/insights-screen.tsx

D. Requirement checklist (engineering commitments from the docs)

Commitment (source)	Status	Evidence
TypeScript strict everywhere (PLAN Phase 0, owner rules)	Met	tsconfig.base.json strict + noUncheckedIndexedAccess; mobile repeats both
800-line ceiling, no barrels (owner rules)	Met	Appendix B and C
Conventional Commits, no agent attribution (owner rules)	Met	75 of 75 subjects match; 0 co-author or agent mentions in bodies
Money as integers (PLAN)	Met	*_minor integer columns, now with CHECKs
Per-user data isolation (README)	Met	user-registry.ts, user-scope.ts, users.test.ts
Offline queue never loses writes (PLAN Phase 7, PRODUCT)	Partial	401 path fixed (CQ-001); CQ-028 and CQ-030 remain
Login rate limit, 5 per 15 min per IP (PLAN Phase 2)	Met in code	limits.test.ts. Production TRUST_PROXY Unverified
Nightly backups, 14 kept, restore check (PLAN Phase 5)	Met in code	integrity check per backup; off-site copy manual (CQ-019)
AI cap and off switch, tested with a fake (PLAN Phase 6)	Met	cap checked before every call; ai-guardrails.test.ts
Every phase ends with tests passing (PLAN)	Met, enforced	check.yml on push to main and on PRs
Account and data deletion (privacy page)	Met	DELETE /auth/account and Settings; edge case CQ-031
Service stays up for all users (implicit; "My data, my server")	Not met	CQ-027

E. File inventory

Workspace	Source files	Source lines	Test files	Test lines
apps/api	100	9,424	30	4,143
apps/mobile	273	20,439	31	1,859
packages/shared	17	3,169	12	1,371
Total	390	33,032	73	7,373

Round 1 had 342 source files with 28,330 lines and 27 test files with 3,421 lines. Migrations are now 0000 to 0005, adding 0003_task_next_id, 0004_constraints and 0005_sync_updated_at. CI lives in .github/workflows/check.yml.

F. Probe scripts (run from the scratch folder against the in-memory test app, then deleted)

```
// probe-tz.ts / probe-crash.ts (CQ-027)
const ctx = await createTestContext();
const other = await ctx.tokenFor("other-user", "other@example.com");
await ctx.request("PATCH", "/settings", { timeZone: "Not/AZone" }, { authorization: `Bearer
${other}` });
await runHourlyJobs(ctx.deps);           // rejects: RangeError
startJobs(ctx.deps);                     // process exits about 10 s later (exit=1)

// probe-check.ts (CQ-030)
const ev = await ctx.request("POST", "/events", { name: "Trip", startsOn: "2026-10-10", endsOn:
"2026-10-12" });
await ctx.request("PATCH", `/events/${ev.body.id}`, { endsOn: "2026-10-01" }); // 500
internal_error

// probe-delete.ts (CQ-031), USER_DATA_DIR and BACKUPS_DIR pointed at a scratch folder
const run = runHourlyJobs(ctx.deps); // takes the user list, then yields
ctx.deps.users.remove(gone);
await run;                           // gone's .db and a backup exist again
```